Motor de recomendación semántica multiperfil

Pipeline completo, estrategias elegidas y resultados (versión 1.4)

Documento de trabajo para el equipo del proyecto

19 de octubre de 2026

Resumen

El motor recomienda películas a un perfil cinéfilo a partir de reseñas de Letterboxd. Las reseñas se vectorizan con un modelo de lenguaje; un *modelo de reglas en dos etapas* predice, desde esas reseñas, cuánto cumple cada película cada afinidad y cada filtro del perfil, y luego calcula la nota global con la forma de la regla global del perfil. Se entrena contra una Verdad Base de 130 películas evaluadas por Gemini, que incluye películas que a la persona no le gustan. Con el modelo de embeddings `multilingual-e5-base`, el orden que produce el modelo se parece al de Gemini con ρ de Spearman = 0.739 en validación cruzada (0.62 con la regresión lineal de las versiones anteriores y 0 al azar), y la nota se equivoca en 0.95 puntos en promedio (1.53 prediciendo siempre el promedio). Agregar las películas que disgustan subió ρ de 0.674 a 0.739. El documento explica cada paso con la estrategia elegida y la descartada, sigue una película de principio a fin (*Manhattan*), resume cómo ejecutar el sistema y los resultados, y propone los pasos siguientes.

Índice

1. El problema y los datos **3**

2. El pipeline **4**

2.1. Filtrado de reseñas (**opción 2**, `src/db/filtered/filter.py`) 5

2.2. Chunks y embeddings de reseñas (**opción 4**, `src/procesing_reviews.py`) 5

2.3. Embeddings del perfil (**opción 3**, `src/procesing_profiles.py`) 6

2.4. Resumen de cada película (*pooling*, `src/pooling.py`) 6

2.5. Rasgos de frases de reseña (`src/pooling.py`) 6

2.6. Modelo de reglas en dos etapas (**opción 7**) 7

2.7. Recomendación (**opción 8**, `src/recommend.py`) 8

2.8. Evaluación (**opción 9**, `src/analisis.py`, `src/metricas.py`) 8

3. Ejemplo guiado: *Manhattan* (Woody Allen, 1979) **9**

4. Cómo ejecutarlo **10**

4.1. Instalación y datos 10

4.2. Menú principal 11

4.3. Variables de entorno 11

4.4. GitHub Actions 11

4.5. Archivos del análisis 11

5. Resultados **12**
5.1. Modelo final y comparación de modelos de embeddings 12
5.2. Dónde se pierde precisión: la etapa 1 13
5.3. Pooling y cantidad de reseñas 14
5.4. Frases de reseña 15
5.5. ¿Cuánto ayudaría tener más películas evaluadas? 15
5.6. Recomendaciones 16

6. Observaciones **16**

7. Pasos propuestos **17**

A. Resumen de decisiones **18**

1. El problema y los datos

Objetivo. Dado un perfil cinéfilo, ordenar películas que la persona no ha evaluado según cuánto le gustarían. Para recomendar importa más *el orden* que la nota exacta, por eso la métrica principal es la correlación de Spearman (ρ) entre el orden del modelo y el orden de la Verdad Base (sección 2.8).

El perfil (`src/db/profiles/perfiles.ejemplo.json`) describe el gusto en lenguaje natural y es genérico: puede tener cualquier cantidad de criterios.

- **Afinidades** (5): lo que la persona valora, p. ej. *Contemplación Inmersiva* o *Ternura y Empatía Radical*. Cada una tiene nombre, descripción y encabezado de planilla.
- **Filtros restrictivos** (5): lo que le molesta, p. ej. *Insoportabilidad Prolongada* (convivir con personajes arrogantes o crueles). Cada uno tiene descripción, severidad, las afinidades que *corrompe* y una **excepción** (cuándo el filtro no aplica). La cadena es **afinidad** ← **filtro** ← **excepción**: el filtro resta, salvo que su excepción lo neutralice.
- **Regla global**: cuatro puntos en texto que explican cómo combinar todo en una nota 0–10.
- **Frases de reseña** (opcional, nuevo en 1.2): frases como las escribiría alguien en Letterboxd, p. ej. *"Oscar bait designed to make you cry"* para *Control Emocional Artificial*.

Las reseñas. Un *scraper* guarda reseñas de Letterboxd en una base SQLite (`reviews.db`). Se publica solo como `reviews.zip` en el *release* en borrador `datos-resenas` del repositorio, visible únicamente para quienes tienen permiso de escritura: el repositorio es público y las reseñas no se deben publicar. Tras el filtrado (sección 2.1) quedan 16 841 reseñas de 180 películas, que se dividen en 62 802 fragmentos (*chunks*).

La Verdad Base. Gemini evalúa cada película con las instrucciones que genera la **opción 6** (`src/loss/instrucciones.py`): el perfil completo en texto, la regla global y el formato de salida. Para cada película entrega un puntaje 0–10 por afinidad y por filtro, si aplica la excepción de cada filtro (0/1) y la nota global. Hay 130 películas evaluadas (`dataset_maestro.csv` → `matriz_perdida.csv`): 110 de la lista de películas preferidas de la persona y 20 agregadas en la versión 1.4.2, la mayoría con notas bajas (*Green Book*, *Bohemian Rhapsody*, *The Blind Side*, *Joker*, entre otras). 129 tienen reseñas y sirven para entrenar (falta *Nickel Boys*), y 58 películas con reseñas aún no tienen nota: son las candidatas a recomendar. Gemini es bastante consistente consigo mismo: al reevaluar 9 películas, la nota global tuvo $\rho = 0.95$ con la evaluación anterior y los puntajes por criterio $\rho = 0.83$ (`evaluaciones_anteriores.csv`).

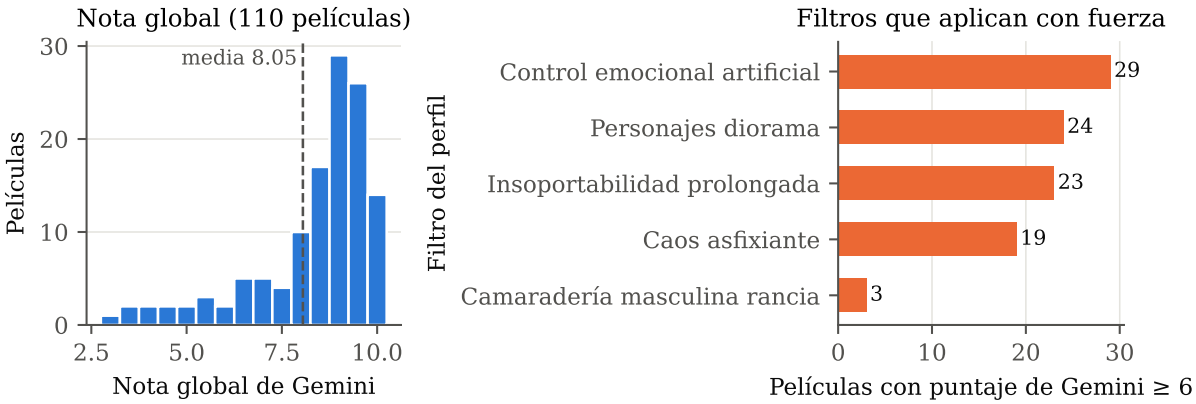


Figura 1: La Verdad Base sigue dominada por películas preferidas (nota media 8.05; 95 de 130 tienen 8 o más), pero las 20 películas agregadas suman 18 con nota bajo 6 (izquierda). Los filtros aplican con fuerza en más películas; *Camaradería Masculina Rancia* sigue siendo el más escaso, con 3 películas con puntaje ≥ 6 (derecha).

2. El pipeline

La figura 2 resume el flujo y la opción del menú (`python src/main.py`) que ejecuta cada paso. Las secciones siguientes explican cada paso: qué hace, la estrategia elegida, la variante básica o alternativa descartada y la evidencia para decidir. Todas las comparaciones se hicieron con validación cruzada repetida (sección 2.8).

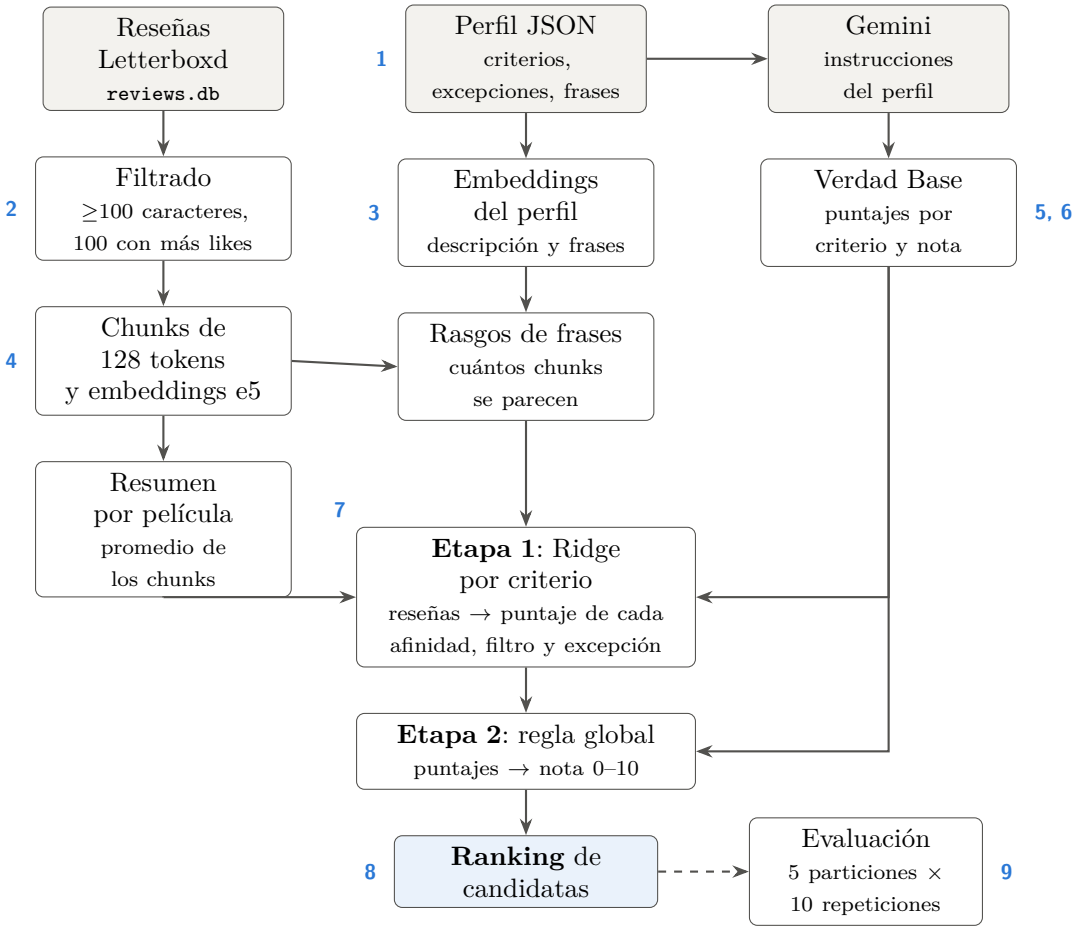


Figura 2: Pipeline. Los números azules son las opciones del menú; la **opción 9** ejecuta de la 2 a la 8 sin preguntas y agrega las comparaciones y el análisis.

2.1. Filtrado de reseñas (opción 2, `src/db/filtered/filter.py`)

Lee la base SQLite con una consulta con funciones de ventana: por cada película toma las reseñas con más *likes*, descartando las de menos de 100 caracteres (suelen ser chistes de una línea que no describen la película), y solo películas con al menos 20 reseñas. Limpia caracteres especiales y deja `film_id` (el *slug* de Letterboxd), título, director, idioma, *likes* y texto. Si el resultado es idéntico al último filtrado no crea otro archivo, para no duplicar embeddings después.

Elegida	hasta **100 reseñas por película**, ordenadas por *likes* (`MAX_RESENIAS_POR_PELICULA`).
Descartadas	200 reseñas: ordena igual o peor y duplica el tiempo de generar embeddings (con MiniLM, 100 ordenó mejor que 200 en las 10 repeticiones, $\Delta\rho = +0.021$, con la Verdad Base anterior). 50 reseñas: no mejora de forma consistente (fig. 5).

2.2. Chunks y embeddings de reseñas (opción 4, `src/procesing_reviews.py`)

Cada reseña se divide con `semchunk` en fragmentos de hasta 128 tokens y cada fragmento se convierte en un vector (*embedding*) con un modelo multilingüe de `sentence-transformers`. Los vectores se guardan en ChromaDB (colección `resenias`); cada modelo en su propia carpeta (`src/db/embeddings/chroma_<modelo>/`) porque sus dimensiones no se pueden mezclar. El identificador de cada chunk incluye película, lote, reseña y número de chunk, así que reprocesar

un lote no duplica. Cuando llega una versión nueva de `reviews.db`, solo se codifican los chunks cuyo texto no está en la base: los demás reutilizan su vector, y el lote anterior se elimina (`src/embeddings_previos.py`). Al agregar las reseñas de 28 películas, e5 codificó 7 117 chunks y reutilizó 55 685 (89 %). Chroma no libera el espacio de lo eliminado; `src/compactar_chroma.py` copia la base a una nueva.

Elegida	`intfloat/multilingual-e5-base` (768 dimensiones), con el prefijo `"query: "` que exige la familia e5. Por defecto desde la versión 1.3.
Básica (descartada)	`paraphrase-multilingual-MiniLM-L12-v2` (384 dimensiones), la de las versiones anteriores: $\rho = 0.718$ contra 0.739 con e5 (tabla 3). Sigue disponible con la variable `MODELO_EMBEDDINGS`.
Alternativa	`paraphrase-multilingual-mpnet-base-v2`: $\rho = 0.721$. Con las películas que disgustan, las diferencias entre modelos se achicaron (antes 0.605, 0.651 y 0.674).
Costo	e5 y mpnet tardan unas 3 veces más por chunk: 1 h 40 min en GitHub Actions para codificar 55 769 chunks desde cero, sin GPU; con la base anterior reutilizada, la corrida completa tarda unos 45 minutos.

2.3. Embeddings del perfil (**opción 3**, `src/procesing_profiles.py`)

Cada afinidad y cada filtro se vectoriza con el mismo modelo, junto a un documento por excepción y uno por cada frase de reseña (colección `perfiles`).

Elegida	vectorizar solo **nombre y descripción** de cada criterio.
Descartada	el texto completo del criterio (con sus afinidades corrompidas y su excepción): acercaba el vector del filtro a los conceptos que se oponen a él. La similitud entre *Insoportabilidad* y *Ternura* bajó de 0.68 a 0.50 al quitar ese texto.

2.4. Resumen de cada película (***pooling***, `src/pooling.py`)

Una película tiene cientos de chunks; la etapa 1 necesita un solo vector por película.

Elegida	**promedio** de los embeddings de todos sus chunks (`POOLING_RESENIAS=media`).
Descartadas	percentil 50, 75, 90 o 95 de cada dimensión del embedding, y promedio más percentiles de la similitud de los chunks con cada término del perfil. Con e5 ninguna ordena mejor que el promedio en 10 repeticiones (la mejor, $\Delta\rho = +0.000$; con 100 o todas las reseñas, los percentiles 75 a 95 bajan entre 0.014 y 0.021, peores en casi todas las repeticiones). Con MiniLM y mpnet algunas mejoran levemente (hasta +0.017), sin repetirse con e5. Con una sola partición el percentil 90 parecía ganar, pero era ruido (fig. 5). Se pueden elegir con `POOLING_RESENIAS=pXX`.

2.5. Rasgos de frases de reseña (`src/pooling.py`)

La descripción de un filtro está escrita como la escribiría un crítico ("control emocional artificial"); las reseñas dicen "Oscar bait". Un experimento previo (`src/experimentos/sondas_filtros.py`) midió, para cada texto, qué fracción de los chunks de cada película se parece mucho a él y la correlacionó con el puntaje de Gemini del filtro (tabla 6): las frases con el vocabulario de las reseñas detectan mejor cuatro de los cinco filtros.

Para cada criterio con `frases_resenia`, y para cada texto t (su descripción y cada frase), se calcula:

- la **fracción de chunks** de la película con similitud coseno con t mayor que el percentil 95 de esa similitud en una muestra de 20 000 chunks de todas las películas (el umbral no usa notas), y
- el **percentil 90** de la similitud de los chunks de la película con t.

Con 3–4 frases por filtro son 8–10 rasgos. El umbral de cada texto se guarda con el modelo para medir igual a las películas que se recomiendan después.

Elegida	**ponderar**: los rasgos se agregan al embedding y, tras estandarizar, se multiplican por $\sqrt{768/k}$ (k = cantidad de rasgos), para que pesen en conjunto lo mismo que las 768 dimensiones del embedding. Con la Verdad Base de 101 películas mejoraba el orden con e5 en las 10 repeticiones ($\Delta\rho = +0.008$). Con las 129 actuales la ganancia casi desaparece: $+0.004 \pm 0.004$, mejor en 6 de 10; con MiniLM empeora levemente (-0.004). Mejora la etapa 1 de tres filtros (*Insoportabilidad* 0.22 → 0.27, *Control* 0.47 → 0.51, *Caos* 0.45 → 0.47) y empeora la de *Camaradería* y *Diorama*.
Descartadas	concatenarlos sin ponderar (Ridge los diluye: +0.001); usar solo las frases, sin el embedding (-0.003); y los controles con solo la descripción (+0.000 y -0.008).
Lectura	las frases aportaban sobre todo cuando faltaban ejemplos de películas con los filtros presentes; con esos ejemplos, el embedding promedio ya recoge buena parte de la señal. Se mantienen porque con e5 no empeoran y ayudan en los filtros con más vocabulario propio, pero conviene revisar las frases de *Camaradería* y *Diorama*.

2.6. Modelo de reglas en dos etapas (opción 7)

Implementado en `src/modelo_reglas.py` y entrenado por `src/scoring.py`.

Etapa 1: de las reseñas a los puntajes. Para cada criterio c (afinidad, filtro o excepción) una regresión Ridge, con estandarización y la regularización elegida por validación cruzada interna ($\alpha \in [10^{-1}, 10^{4}]$), predice el puntaje de Gemini a partir del embedding promedio de la película y, si el criterio tiene frases, de sus rasgos. Las predicciones se recortan a $[0, 10]$ (a $[0, 1]$ para las excepciones). Son 11 579 coeficientes regularizados en total (768 + rasgos por criterio).

Etapa 2: de los puntajes a la nota. La regla global del perfil se traduce a una fórmula con parámetros que se aprenden de la Verdad Base:

$$\text{nota} = b + \sum_{j \in \text{afinidades}} w_j \, A_j \; - \sum_{k \in \text{filtros}} d_k \, \sigma(1.5 \, (F_k - u_k)) \, (1 - E_k), \tag{1}$$

donde A_j, F_k y E_k son los puntajes de afinidades, filtros y excepciones, $w_j \in [0, 2]$ el peso de cada afinidad, u_k el umbral desde el que el filtro k "se activa", d_k cuánto resta como máximo, y σ la sigmoide, que hace suave el umbral para poder optimizarlo. La excepción ($E_k = 1$) anula el descuento. Se ajusta con L-BFGS-B minimizando el error cuadrático, con tres inicios y una penalización L2 pequeña (10^{-3}) sobre los descuentos. Tiene $1 + n_A + 2n_F$ parámetros: 16 para este perfil, y la misma forma sirve para cualquier perfil. La etapa 2 se entrena con las 130 películas de la Verdad Base, porque no necesita reseñas. Con las películas que disgustan, la regla aprendió descuentos que antes no tenía datos para aprender, como *Camaradería* y *Caos*.

La regla aprendida con e5 es:

- Nota base = 3.75 + 0.06 · Resistencia Femenina + 0.12 · Contemplación Inmersiva + 0.16 · Ternura y Empatía Radical + 0.13 · Humanismo Social + 0.21 · Vanguardia y Simbolismo
- Si Camaradería Masculina Rancia supera 3.7 y su excepción no aplica: resta hasta 1.73 puntos.
- Si Insoportabilidad Prolongada supera 5.3 y su excepción no aplica: resta hasta 1.25 puntos.
- Si Control Emocional Artificial supera 7.8 y su excepción no aplica: resta hasta 1.52 puntos.
- Si Personajes Diorama supera 3.3 y su excepción no aplica: resta hasta 0.18 puntos.
- Si Caos Asfixiante supera 8.6 y su excepción no aplica: resta hasta 1.54 puntos.

Elegida	**modelo de reglas** de dos etapas con la regla simple (1).
Básica (descartada)	la **regresión lineal** de las versiones anteriores: similitud coseno entre el embedding de la película y el de cada término del perfil, más un producto por relación (afinidad × filtro, filtro × excepción), 21 variables. Ordena peor ($\rho = 0.617$ contra 0.739 con e5) y se equivoca más en la nota (MAE 1.16 contra 0.95). La **opción 9** la sigue calculando como comparación.
Alternativas de etapa 2	regla con *pisos* (una afinidad alta asegura una nota mínima) y *corrupción* (el filtro reduce las afinidades que corrompe): MAE 0.293 contra 0.294 con casi el doble de parámetros (31); quedan como opciones desactivadas. También se probaron regresión lineal (0.333), GAM, *gradient boosting* con y sin restricciones monótonas y *random forest* (peores) y la regla corregida con *boosting* (0.280, apenas mejor y mucho menos legible). Estas comparaciones se hicieron con las 110 películas de la Verdad Base anterior.

2.7. Recomendación (**opción 8**, `src/recommend.py`)

Carga el modelo guardado por la **opción 7** (`src/loss/modelo_reglas_<modelo>.joblib`, con el pooling, las frases y sus umbrales) y ordena todas las películas con reseñas por la nota predicha. Las que ya tienen nota de Gemini aparecen para comparar.

2.8. Evaluación (**opción 9**, `src/analisis.py`, `src/metricas.py`)

- **Validación cruzada**: cada película se predice con un modelo que no la vio. 5 particiones, repetidas con 10 semillas distintas; se informa el promedio y la desviación. Con unas 100 películas, el ρ de una sola partición variaba ± 0.05, tanto como las diferencias que se quieren medir; con 129, ± 0.02.
- **Comparaciones pareadas**: cada variante se compara contra una referencia *en las mismas particiones* ($\Delta\rho$), que varía mucho menos, y se cuenta en cuántas de las 10 repeticiones la supera.
- **Métricas**: ρ de Spearman (principal: el orden completo); NDCG@10 (si las 10 primeras según el modelo son las mejores según Gemini, en ese orden, con ganancia $y - \min y$); precisión@10 (fracción de las 10 primeras que están entre las 10 mejores, contando empates en el corte), y MAE de la nota. La línea base (predecir el promedio) se informa con los valores esperados de un orden al azar.

Elegida	ρ de Spearman como métrica principal, promedio de 10 repeticiones.
Descartada	MAE como métrica principal y una sola partición: favorecía variantes que no ordenan mejor (las frases de reseña casi no cambian el MAE pero sí el orden) y hacía que variantes iguales parecieran distintas.

3. Ejemplo guiado: *Manhattan* (Woody Allen, 1979)

Manhattan es una de las pocas películas de la lista con nota baja (Gemini: 5.0) y la razón es un filtro: la película obliga a convivir con un protagonista cuya relación con una adolescente de 17 años y cuyo aire de superioridad intelectual encajan en *Insoportabilidad Prolongada* (Gemini: 8 de 10, sin excepción). Además Gemini le da puntajes bajos en tres afinidades. Es un caso difícil y por eso instructivo: muestra cada paso y también dónde falla el modelo. Para que el ejemplo sea honesto, el modelo de esta sección se entrenó **con todas las demás películas y sin Manhattan**, como si fuera una candidata nueva.

Paso 1: reseñas. El filtrado toma sus 100 reseñas con más *likes* y al menos 100 caracteres, en inglés, portugués, persa, árabe y otros idiomas. Entre ellas hay críticas al personaje, que mencionan su edad y lo tratan de *pretentious* y *creep*, pero son una minoría frente a las que elogian la fotografía en blanco y negro y la música de Gershwin.

Paso 2: chunks y embeddings. Las reseñas se dividen en 287 chunks de hasta 128 tokens y cada uno se convierte en un vector de 768 dimensiones con e5. El promedio de esos vectores es la representación de la película. En el promedio, las críticas de la minoría se diluyen: para eso están los rasgos de frases.

Paso 3: rasgos de frases. Para *Insoportabilidad Prolongada*, la tabla 1 muestra qué fracción de los chunks de Manhattan supera el umbral de similitud con cada texto. Con la frase *"I couldn't stand the protagonist, he is a pretentious narcissist and a creep"*, el 11.5 % de sus chunks la supera, contra 4.2 % en la película mediana: Manhattan está en el percentil 95 de las 101 películas. Con la descripción del perfil, escrita en otro registro, queda en el percentil 26. Las frases sí encuentran el problema.

Texto	Manhattan	Mediana	Percentil
descripción	2.1 %	3.8 %	26
the main character is insufferable, arrogant and smug	7.3 %	4.6 %	85
I couldn't stand the protagonist, he is a pretentious narcissist and a creep	11.5 %	4.2 %	95
an unlikable, annoying character you have to spend the whole movie with	5.6 %	4.7 %	65
un protagonista insoportable, arrogante y pedante	6.3 %	4.1 %	67

Tabla 1: Rasgos de frases de *Insoportabilidad Prolongada* para Manhattan: fracción de sus chunks sobre el umbral de cada texto, la misma fracción en la película mediana y el percentil de Manhattan entre las 101 películas.

Pasos 4 y 5: etapa 1 y regla global. La etapa 1 predice los 14 puntajes desde el embedding y los rasgos, y la regla (1) los combina (tabla 2). La nota base es $b + \sum_j w_j A_j = 8.31$ y los filtros restan 0.03 en total: la nota predicha queda en **8.28**, lejos del 5.0 de Gemini.

Criterio	Gemini	Etapa 1 (predicho)	Parámetros de la regla	Aporte
Sesgo				3.74
Resistencia Femenina	2.0	7.0	$w = 0.10$	+0.69
Contemplación Inmersiva	6.0	7.0	$w = 0.11$	+0.74
Ternura y Empatía Radical	2.0	7.2	$w = 0.15$	+1.09
Humanismo Social	3.0	6.2	$w = 0.13$	+0.79
Vanguardia y Simbolismo	6.0	6.3	$w = 0.20$	+1.26
Camaradería Masculina Rancia	9.0 (exc. 1)	0.3 (exc. 0.11)	$u = 4.9$, $d = 0.05$, act. 0.00	0.00
Insoportabilidad Prolongada	8.0 (exc. 0)	3.0 (exc. 0.36)	$u = 5.6$, $d = 1.68$, act. 0.01	−0.02
Control Emocional Artificial	3.0 (exc. 0)	2.9 (exc. 0.00)	$u = 8.8$, $d = 2.16$, act. 0.00	0.00
Personajes Diorama	4.0 (exc. 0)	3.3 (exc. 0.01)	$u = 7.1$, $d = 0.65$, act. 0.00	0.00
Caos Asfixiante	0.0 (exc. 0)	0.0 (exc. 0.00)	$u = 2.2$, $d = 0.24$, act. 0.04	−0.01
Nota predicha (Gemini: 5.0)				**8.28**

Tabla 2: Cálculo de la nota de Manhattan con el modelo entrenado sin ella. Para los filtros, "act." es $\sigma(1.5(F_k - u_k))(1 - E_k)$, la fracción del descuento d_k que se aplica; "exc." es la excepción.

Dónde falla. La etapa 2 hace lo que dice la regla, pero recibe puntajes equivocados de la etapa 1:

- **Afinidades**: la etapa 1 predice cerca de 7 para todas, el valor típico de la lista, mientras Gemini da 2 o 3 en tres de ellas. Con 101 películas casi todas bien evaluadas, Ridge aprende a quedarse cerca del promedio.
- **El filtro**: aunque el rasgo de la frase está en el percentil 95, la etapa 1 predice 3.0 para *Insoportabilidad*, bajo su umbral de 5.6, y el descuento no se activa. Hay 19 películas con ese filtro alto: pocas para que Ridge le dé tanto peso al rasgo.
- **Camaradería Masculina Rancia**: Gemini le da 9, pero con su excepción aplicada; Manhattan es la *única* película de la lista con ese filtro alto, así que al sacarla del entrenamiento no queda ningún ejemplo.

En el ranking. En la validación cruzada de 10 repeticiones, Manhattan queda en el puesto 84 de 101 (nota predicha 8.32; en Gemini es la 100). El caso contrario es *Marty Supreme* (Gemini 5.5, puesto 98): el modelo la deja última (101 de 101, nota 6.91). *Amélie* (4.5, la peor nota con reseñas) queda en el puesto 81: sus reseñas describen lo dulce y excéntrico como virtud, mientras el perfil lo considera *Control Emocional Artificial*. Los errores grandes se concentran en películas cuyo defecto, según el perfil, las reseñas no describen como defecto, o describen en pocas reseñas. Más películas con notas bajas en la Verdad Base atacarían ambos problemas (sección 7).

4. Cómo ejecutarlo

4.1. Instalación y datos

```
pip install -r requirements.txt
# perfil de ejemplo:
cp src/db/profiles/perfiles.ejemplo.json src/db/profiles/perfiles.json
python src/main.py
```

1. Descargar del *release* en borrador `datos-resenas` el archivo `reviews.zip` y descomprimirlo en `src/db/raw/` (queda `reviews.db`).
2. **Embeddings**: generarlos en CPU con e5 (opciones 2 a 4) tarda varias horas. El mismo *release* tiene la base ya generada, `embeddings-multilingual-e5-base-100-<huella>.zip`:

descomprimirla en la raíz del repositorio. Crea `src/db/embeddings/chroma_multilingual-e5-base/` y el CSV filtrado correspondiente; las opciones 2 y 4 reconocen que ya están procesados.

3. Ejecutar la **opción 9** (todo el pipeline y el análisis), o las opciones 3, 5, 7 y 8 por separado.

4.2. Menú principal

Opción	Qué hace
1	Crear o editar perfiles: criterios, excepciones, encabezados, regla global y frases de reseña.
2	Filtrar las reseñas crudas (`src/db/raw/*.db` o `*.csv`).
3	Embeddings del perfil (descripciones, excepciones y frases).
4	Embeddings de los lotes de reseñas pendientes.
5	Cargar la Verdad Base desde `dataset_maestro.csv` y las evaluaciones adicionales.
6	Listar películas con reseñas sin nota y generar la plantilla y las instrucciones para Gemini.
7	Entrenar el modelo de reglas; informa ρ, NDCG@10, precisión@10 y MAE en validación cruzada y las reglas aprendidas.
8	Ranking de recomendaciones con el modelo guardado.
9	Ejecutar de la 2 a la 8 sin preguntas y guardar el análisis en `analisis/<fecha>_<modelo>/`.
10	Salir.

4.3. Variables de entorno

Variable	Uso
`MODELO_EMBEDDINGS`	Modelo de embeddings (por defecto `intfloat/multilingual-e5-base`).
`MAX_RESENIAS_POR_PELICULA`	Reseñas por película (por defecto 100).
`POOLING_RESENIAS`	`media` (por defecto) o `pXX`. El modelo guardado recuerda cuál usó.

4.4. GitHub Actions

- `analisis.yml`: ejecuta la **opción 9** con MiniLM, mpnet y e5 en paralelo y publica los resultados en la rama `resultados-analisis` (solo números, nunca texto de reseñas). Reutiliza los embeddings del caché; si cambió `reviews.zip`, parte de la base de la versión anterior y solo codifica los chunks nuevos. Al terminar compacta la base, y el job de e5 la publica en el *release* en borrador. Corre al cambiar el workflow, el perfil de ejemplo, `dataset_maestro.csv`, `analisis.py`, `modelo_reglas.py` o `pooling.py`. Lanzado a mano con la opción *calcular también los datos del informe*, el job de e5 corre además `docs/informe/calculos.py`.
- `sondas.yml`: el experimento de frases sonda de la sección 2.5.
- `tests.yml`: en cada *push*, 37 pruebas unitarias con datos sintéticos (`python -m pytest tests`) y la revisión estática del código con `pyflakes`.

4.5. Archivos del análisis

`resumen.md` (todas las tablas), `peliculas.csv` (por película: similitudes, puntajes predichos, notas y errores), `ranking.csv`, `reglas.json`, `pooling.csv`, `frases_resenia.csv`, `pesos.csv` y

`similitud_perfil.csv`; con los datos del informe, también la carpeta `informe/` (predicciones por película, ejemplo guiado y curva de aprendizaje). Para regenerar este informe se copian esos archivos a `docs/informe/datos/` y se ejecutan `python docs/informe/tablas.py`, `python docs/informe/figuras.py` y `latexmk -pdf informe.tex` (este último desde `docs/informe/`).

5. Resultados

Las tablas 3 a 8 y las figuras 5 y 6 vienen de la corrida del workflow de análisis del 19 de octubre (commit `25f7d2b`), publicada en la rama `resultados-analisis`. La curva de aprendizaje, la dispersión y el ejemplo guiado se calcularon con `docs/informe/calculos.py` en una corrida posterior del mismo workflow, con los mismos datos. La tabla de frases sonda (tabla 6) es de la Verdad Base anterior. Los datos agregados de cada tabla y figura están en `docs/informe/datos/`.

5.1. Modelo final y comparación de modelos de embeddings

Embeddings	Modelo	ρ Spearman	NDCG@10	Precisión@10	MAE
MiniLM	**Reglas + frases (op. 7 y 8)**	**0.718** ± 0.011	0.946	0.310	1.041
	Regresión lineal anterior	0.585	0.932	0.400	1.273
	Regresión lineal con Ridge	0.633	0.931	0.200	1.148
mpnet-base	**Reglas + frases (op. 7 y 8)**	**0.721** ± 0.013	0.956	0.360	1.004
	Regresión lineal anterior	0.691	0.941	0.300	1.133
	Regresión lineal con Ridge	0.700	0.947	0.400	1.092
e5-base	**Reglas + frases (op. 7 y 8)**	**0.739** ± 0.012	0.959	0.310	0.945
	Regresión lineal anterior	0.617	0.917	0.200	1.157
	Regresión lineal con Ridge	0.653	0.946	0.300	1.084
Orden al azar / predecir el promedio		0	0.761	0.101	1.531

Tabla 3: Validación cruzada (129 películas). El modelo de reglas es el promedio de 10 repeticiones ± su desviación; las regresiones lineales, una partición. En negrita, el modelo que usan las opciones 7 y 8.

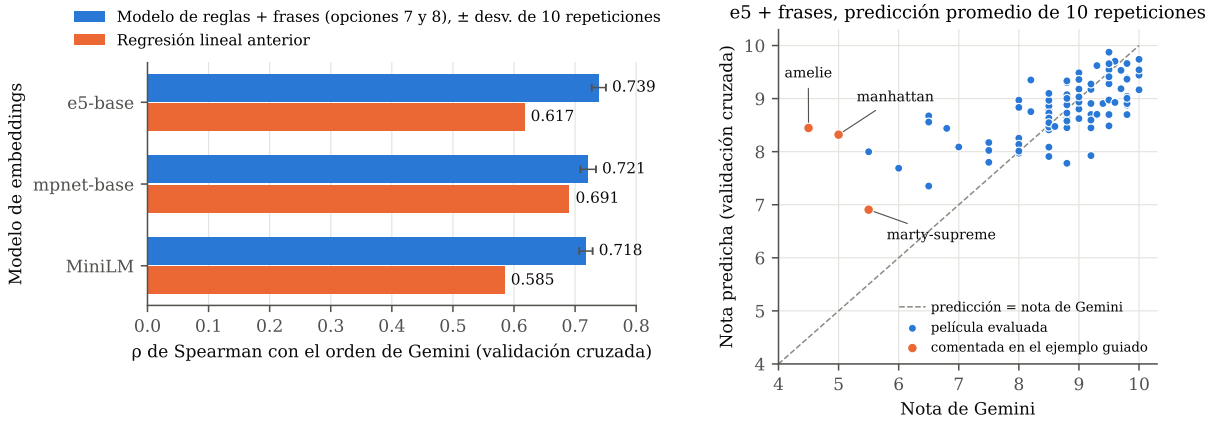


Figura 3: Izquierda: ρ de Spearman por modelo de embeddings (datos de la tabla 3). Derecha: nota de Gemini contra la nota predicha en validación cruzada con e5; en naranja, películas comentadas en el texto. El modelo comprime las notas hacia el promedio: sube las peores y baja las mejores, pero conserva buena parte del orden.

5.2. Dónde se pierde precisión: la etapa 1

Con los puntajes reales de Gemini, la etapa 2 explica el 94 % de la varianza de la nota global (tabla 4; con la Verdad Base anterior, 88 %). Con los puntajes que predice la etapa 1 desde las reseñas, el modelo completo explica el 53 % (antes 39 %). El cuello de botella sigue siendo predecir los criterios desde las reseñas, sobre todo los filtros (fig. 4), aunque con las películas que disgustan el R^2 medio de los filtros subió de 0.22 a 0.34 y el de las afinidades de 0.45 a 0.53.

Entrada de la etapa 2	ρ Spearman	R^2	MAE
Puntajes reales de Gemini (110 películas)			
Regla con condiciones (16 parámetros)	0.943	0.937	0.359
Regresión lineal sobre los puntajes	0.915	0.929	0.410
Predecir el promedio	0.000	-0.011	1.524
Modelo completo (etapa 1 desde reseñas + regla), e5	0.739	0.532	0.945

Tabla 4: Etapa 2 por separado (validación cruzada sobre los puntajes de Gemini) y modelo completo.

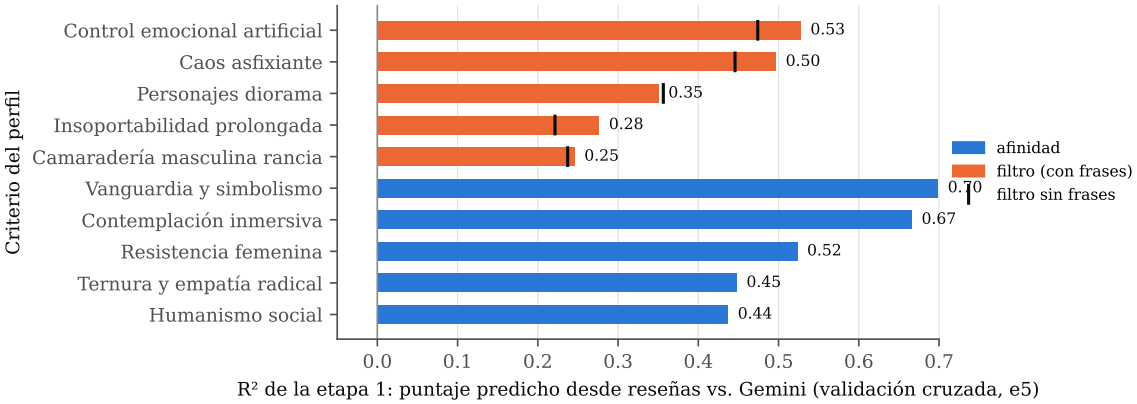


Figura 4: R^2 de la etapa 1 por criterio (e5). Las afinidades se predicen mejor que los filtros. Las frases de reseña (barra) mejoran tres filtros respecto del modelo sin frases (marca negra) y empeoran *Camaradería* y *Diorama*. *Camaradería Masculina Rancia* tiene solo 3 películas con puntaje alto.

5.3. Pooling y cantidad de reseñas

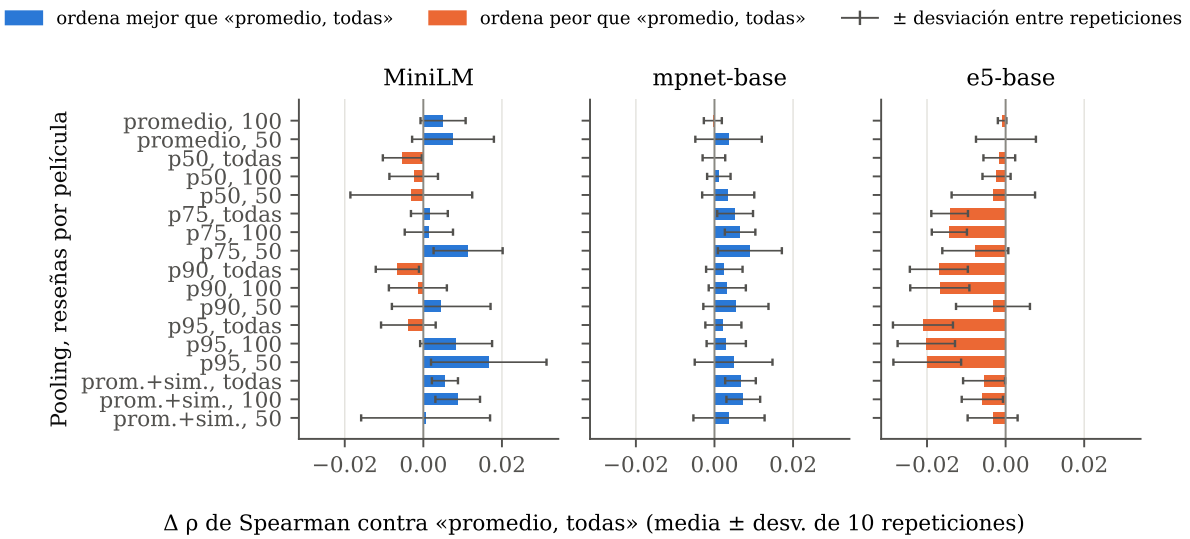


Figura 5: Diferencia de ρ de cada forma de resumir las reseñas contra el promedio de todas (media ± desviación de 10 repeticiones pareadas). Con mpnet y e5 "todas" son 100 reseñas. Ninguna variante mejora de forma consistente con los tres modelos; con e5, el promedio es el mejor y los percentiles altos empeoran.

Variante	MiniLM		mpnet-base		e5-base	
	$\Delta\rho$	mejora	$\Delta\rho$	mejora	$\Delta\rho$	mejora
promedio, todas (elegida)	$\rho = 0.722$ (ref.)		$\rho = 0.722$ (ref.)		$\rho = 0.735$ (ref.)	
promedio, 100	+0.005 ± 0.006	8/10	-0.000 ± 0.002	5/10	-0.001 ± 0.001	1/10
promedio, 50	+0.008 ± 0.010	7/10	+0.004 ± 0.008	7/10	+0.000 ± 0.008	6/10
percentil 50, todas	-0.005 ± 0.005	3/10	-0.000 ± 0.003	4/10	-0.002 ± 0.004	4/10
percentil 50, 100	-0.002 ± 0.006	3/10	+0.001 ± 0.003	6/10	-0.002 ± 0.004	4/10
percentil 50, 50	-0.003 ± 0.015	5/10	+0.003 ± 0.007	8/10	-0.003 ± 0.011	5/10
percentil 75, todas	+0.002 ± 0.005	6/10	**+0.005** ± 0.005	9/10	-0.014 ± 0.005	0/10
percentil 75, 100	+0.001 ± 0.006	6/10	**+0.007** ± 0.004	9/10	-0.014 ± 0.004	0/10
percentil 75, 50	**+0.011** ± 0.009	9/10	**+0.009** ± 0.008	10/10	-0.008 ± 0.008	1/10
percentil 90, todas	-0.007 ± 0.005	1/10	+0.003 ± 0.005	7/10	-0.017 ± 0.007	0/10
percentil 90, 100	-0.001 ± 0.007	4/10	+0.003 ± 0.005	7/10	-0.017 ± 0.008	1/10
percentil 90, 50	+0.005 ± 0.013	7/10	+0.005 ± 0.008	7/10	-0.003 ± 0.009	4/10
percentil 95, todas	-0.004 ± 0.007	3/10	+0.002 ± 0.005	7/10	-0.021 ± 0.008	0/10
percentil 95, 100	+0.008 ± 0.009	7/10	+0.003 ± 0.005	7/10	-0.020 ± 0.007	0/10
percentil 95, 50	**+0.017** ± 0.015	9/10	+0.005 ± 0.010	7/10	-0.020 ± 0.009	0/10
promedio + sim., todas	**+0.005** ± 0.003	10/10	**+0.007** ± 0.004	10/10	-0.005 ± 0.005	1/10
promedio + sim., 100	**+0.009** ± 0.006	9/10	**+0.007** ± 0.004	10/10	-0.006 ± 0.005	0/10
promedio + sim., 50	+0.001 ± 0.016	6/10	+0.004 ± 0.009	6/10	-0.003 ± 0.006	3/10

Tabla 5: Datos de la figura 5: $\Delta\rho$ media ± desviación y repeticiones (de 10) en que la variante supera a la referencia. En negrita, mejoras en al menos 9 de 10.

5.4. Frases de reseña

Filtro	Descripción	Mejor frase	Léxico	Frase con mejor ρ
Camaradería Masculina Rancia	0.01	0.08	-0.03	*"humor machista y sexista, las mujeres son un chiste"*
Insoportabilidad Prolongada	0.39	0.53	0.55	*"I couldn't stand the protagonist, he is a pretentious narcissist and a creep"*
Control Emocional Artificial	-0.04	0.40	0.16	*"Oscar bait designed to make you cry"*
Personajes Diorama	0.03	0.28	0.17	*"style over substance, the characters feel like dolls in a dollhouse"*
Caos Asfixiante	0.30	0.17	0.34	*"overstimulating, loud and relentless, too much happening at once"*

Tabla 6: Experimento de frases sonda (MiniLM, 101 películas de la Verdad Base anterior): ρ de Spearman entre la fracción de chunks muy similares a cada texto y el puntaje de Gemini del filtro, y la correlación de un conteo de palabras clave (léxico). La descripción del perfil, escrita como la escribiría un crítico, detecta mal *Control* y *Diorama*.

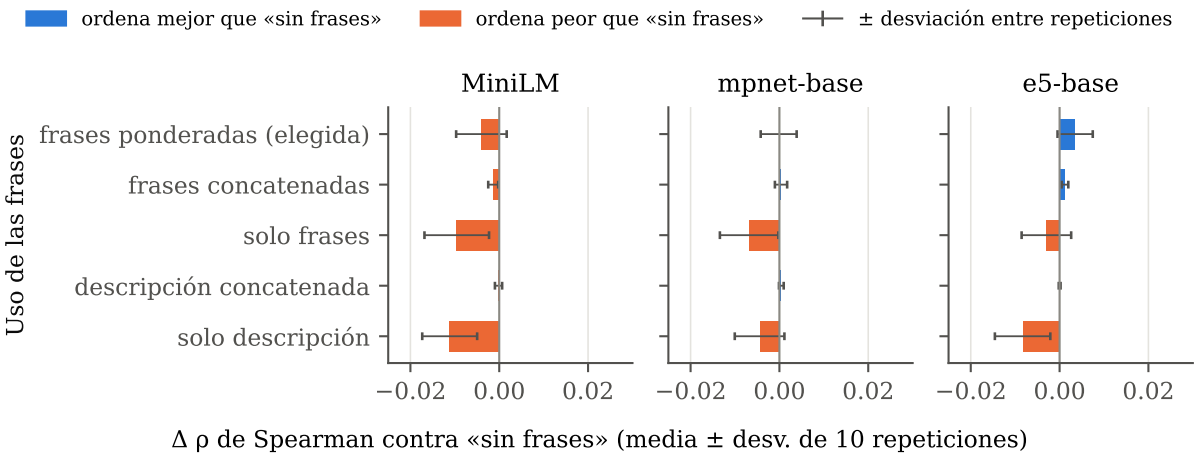


Figura 6: Diferencia de ρ de cada forma de usar las frases contra el modelo sin frases (10 repeticiones pareadas). Con la Verdad Base actual la ganancia es pequeña: con e5, las frases ponderadas mejoran en 6 de 10 repeticiones (+0.004); con la anterior mejoraban en las 10 (+0.008).

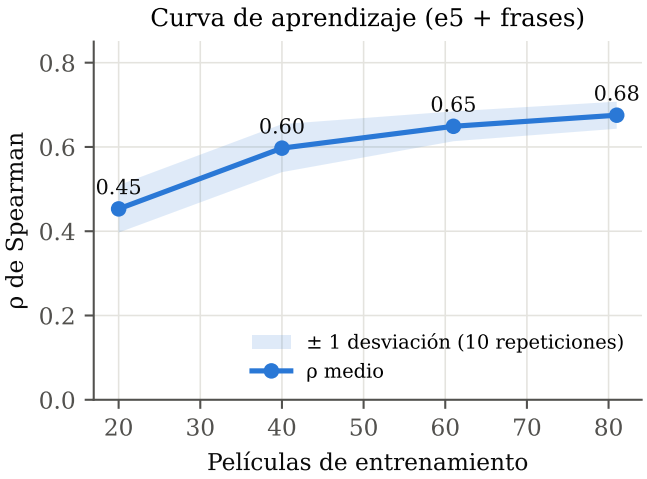
Variante	MiniLM		mpnet-base		e5-base	
	$\Delta\rho$	mejora	$\Delta\rho$	mejora	$\Delta\rho$	mejora
sin frases (referencia)	$\rho = 0.722$ (ref.)		$\rho = 0.722$ (ref.)		$\rho = 0.735$ (ref.)	
frases ponderadas (elegida)	-0.004 ± 0.006	2/10	-0.000 ± 0.004	5/10	+0.004 ± 0.004	6/10
frases concatenadas	-0.001 ± 0.001	1/10	+0.000 ± 0.001	4/10	**+0.001** ± 0.001	9/10
solo frases	-0.010 ± 0.007	2/10	-0.007 ± 0.007	3/10	-0.003 ± 0.006	2/10
descripción concat. (control)	-0.000 ± 0.001	3/10	+0.000 ± 0.001	8/10	+0.000 ± 0.000	4/10
solo descripción (control)	-0.011 ± 0.006	0/10	-0.004 ± 0.006	1/10	-0.008 ± 0.006	2/10

Tabla 7: Datos de la figura 6. En negrita, mejoras en al menos 9 de 10 repeticiones.

5.5. ¿Cuánto ayudaría tener más películas evaluadas?

La curva de aprendizaje (fig. 7) entrena con una fracción de las películas de cada partición y predice el resto. ρ sube de 0.45 con 20 películas a 0.60 con 40, 0.65 con 61 y 0.675 con 81: todavía

sube, pero cada vez menos (+0.026 entre 61 y 81). Más películas parecidas a las actuales darían una mejora moderada; películas con notas bajas y con los filtros presentes deberían rendir más, porque aportan justo la variedad que falta (sección 7).



Películas	ρ Spearman
20	0.453 ± 0.056
40	0.597 ± 0.057
61	0.649 ± 0.035
81	0.675 ± 0.032

Figura 7: Curva de aprendizaje (e5 + frases, 5 particiones × 10 repeticiones): ρ medio ± desviación según la cantidad de películas de entrenamiento.

5.6. Recomendaciones

#	Película	Nota del modelo	Nota lineal anterior
1	`taipei-suicide-story`	9.83	10.31
2	`the-passion-of-joan-of-arc`	9.80	8.38
3	`2046`	9.75	8.38
4	`sentimental-value`	9.72	7.89
5	`chungking-express`	9.59	8.59
6	`happy-together`	9.51	9.19
7	`under-the-skin`	9.48	9.47
8	`portrait-of-a-lady-on-fire`	9.40	8.48
9	`train-dreams`	9.22	9.77
10	`the-seventh-seal`	9.20	10.39

Tabla 8: Las 10 mejores candidatas (sin nota de Gemini) según el modelo de reglas con e5, y la nota que les daba la regresión lineal anterior.

6. Observaciones

1. **Las películas que disgustan fueron la mejora más grande.** Con 20 películas más, casi todas con notas bajas, y las reseñas de las 28 que no tenían, ρ subió de 0.674 a 0.739, más que cualquier cambio de modelo (de MiniLM a e5 fueron +0.07 con la Verdad Base anterior y +0.02 con la actual). La curva de aprendizaje anterior predecía unos +0.02 por 20 películas parecidas a las de la lista. El MAE subió de 0.61 a 0.95 porque las notas ahora tienen mucho más rango: comparado con predecir el promedio, el error bajó de 72 % a 62 %.
2. **La Verdad Base sigue cargada a las preferidas** (95 de 130 con 8 o más) y *Camaradería Masculina Rancia* tiene solo 3 películas con puntaje alto, pocas para aprender a detectarla.
3. **La Verdad Base es la lectura de Gemini, no la nota de la persona.** El modelo aprende a imitar cómo Gemini interpreta el perfil. Gemini es consistente consigo mismo (al

reevaluar 9 películas, $\rho = 0.95$ en la nota global), así que el 0.74 del modelo todavía está lejos del techo. Falta medir cuánto se parece la lectura de Gemini al gusto real.

4. **El cuello de botella es la etapa 1** (tabla 4): la regla explica bien la nota a partir de los puntajes, pero predecir los puntajes desde reseñas es difícil, en especial los filtros, que suelen estar en una minoría de reseñas y se diluyen en el promedio.

5. **El ruido de evaluación es del tamaño de las mejoras.** Una sola partición de la validación cruzada varía ± 0.05 en ρ; varias conclusiones tempranas (el percentil 90, el ranking de modelos con una partición) se revirtieron al repetir. Toda decisión debe tomarse con comparaciones pareadas repetidas.

6. **La etapa 2 tiene óptimos locales parecidos**: diferencias de redondeo pueden llevar al optimizador a otra solución con casi el mismo error; afecta ρ en ~ 0.004.

7. **Las frases se eligieron después de leer reseñas** de algunas películas de la Verdad Base, así que su ganancia puede ser algo optimista. Con las películas nuevas, que no se usaron para escribirlas, la ganancia bajó a $+0.004$.

8. **Los chunks en otros idiomas** (13 % de las reseñas) son el 35 % de los más similares a las frases (*hubness* del modelo multilingüe), pero usar solo inglés no cambió las correlaciones (± 0.04).

9. **Privacidad**: el repositorio es público. Las reseñas y las bases de embeddings (que contienen fragmentos de reseñas) solo van al *release* en borrador; los resultados publicados son agregados e identificadores.

7. Pasos propuestos

1. **Seguir agregando películas que disgustan.** Las primeras 20 dieron la mayor mejora hasta ahora. Conviene sumar otras 20–30, eligiendo a propósito casos de *Camaradería Masculina Rancia* (solo 3) y *Personajes Diorama*, y descargar sus reseñas con el *scraper* antes de evaluarlas.

2. **Tres perfiles más.** Es la premisa del proyecto (multiperfil) y lo más barato de escalar: los embeddings de reseñas no dependen del perfil, así que un perfil nuevo solo necesita su JSON, la **opción 3** y la evaluación de Gemini de las mismas películas. Permite comprobar que la regla genérica se adapta a perfiles con otra cantidad de criterios y comparar la dificultad entre perfiles. Idealmente, perfiles con gustos contrastantes (p. ej. uno que valore el cine de acción o la comedia) para que las mismas películas tengan notas distintas.

3. **Evaluar las 58 candidatas** con la **opción 6**: pasaría de 129 a unas 187 películas de entrenamiento. La curva de aprendizaje (fig. 7) indica cuánto puede rendir.

4. **Validar contra notas reales** de la persona (su diario de Letterboxd) y ampliar la medición de la consistencia de Gemini a unas 20 películas (hoy son 9).

5. **Revisar las frases de reseña** de *Camaradería* y *Diorama*, que hoy empeoran la etapa 1 de esos filtros.

6. **Experimentos de modelo**, con ganancia más incierta: modelos de embeddings más grandes, como `multilingual-e5-large` o `bge-m3` (unas 5 h en Actions); entrenar la etapa 2 optimizando el orden en vez del error, y agregar frases de reseña a las afinidades.

A. Resumen de decisiones

Paso	Elegida	Descartada	Evidencia
Reseñas por película	100 con más *likes*, $\geq$100 caracteres	200 (igual o peor, el doble de lento); 50 (no mejora)	$\Delta\rho$ con MiniLM: 100 vs. 200, +0.021 (10/10)
Modelo de embeddings	e5-base (768 dim.)	MiniLM (básica), mpnet-base	ρ 0.739 vs. 0.718 y 0.721
Embedding del perfil	nombre + descripción	texto completo del criterio	similitud filtro–afinidad 0.68 $\rightarrow$ 0.50
Pooling	promedio	percentiles 50–95, promedio + similitudes	con e5 ninguna mejora ($\leq$ +0.000)
Frases de reseña	ponderadas junto al embedding	concatenadas, solo frases, solo descripción	+0.004 en 6/10 (antes +0.008 en 10/10)
Modelo	reglas en dos etapas	regresión lineal (básica), Ridge	ρ 0.739 vs. 0.617 y 0.653
Etapa 2	regla simple (16 parámetros)	pisos y corrupción (31), GAM, *boosting*, *random forest*	MAE 0.294 vs. 0.293 con el doble de parámetros; árboles peores (110 películas)
Métrica	ρ de Spearman, 10 repeticiones	MAE, una partición	una partición varía $\pm$0.02–0.05

Tabla 9: Estrategias elegidas y descartadas en cada paso del pipeline.